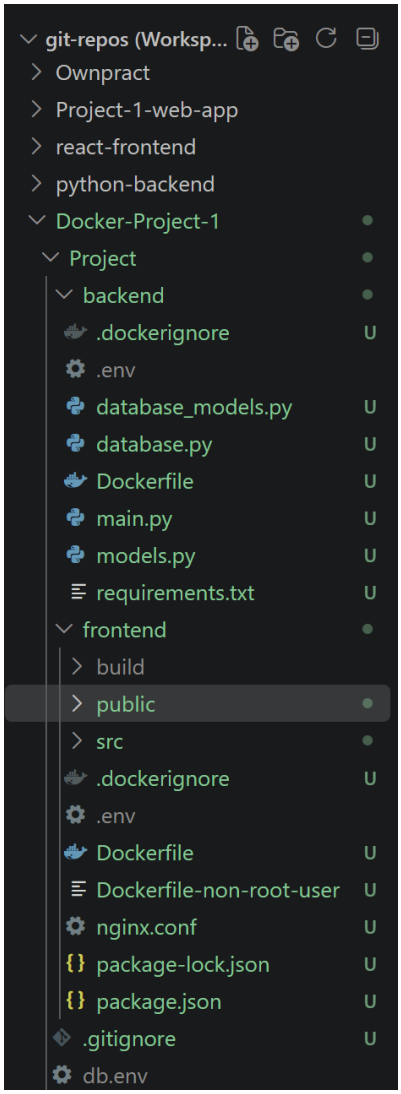
Project-1
Deployment of a 3-Tier Application on AWS EC2 using Docker and Docker Compose

Objective:

The objective of this project is to deploy a 3-tier application (React + FastAPI + PostgreSQL) on an AWS EC2 instance using Docker and Docker Compose. The application (Relentless Tracker, a product inventory manager) should be accessible through the EC2 Public IP address and all the services should communicate successfully.

Project Structure:



Project folder layout (VS Code Explorer): backend, frontend, db.env, docker-compose.yaml.

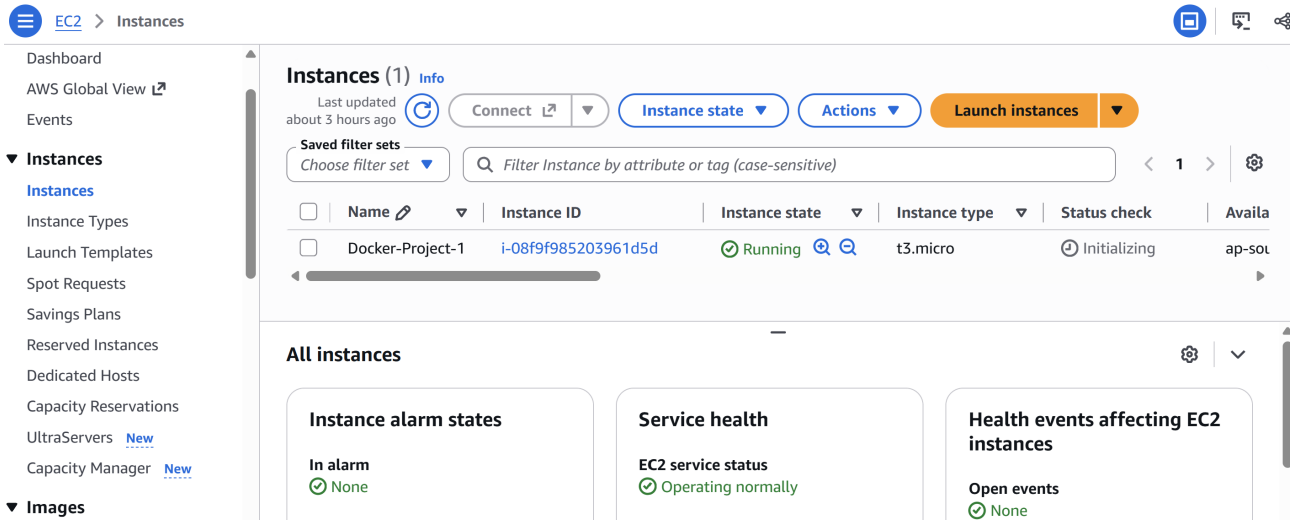
Project Architecture:

User ---> Frontend (React, served by Nginx) ---> Backend (FastAPI) ---> Database (PostgreSQL)

Only the frontend container is published on the EC2 host (port 80). The backend and the database are reachable only on the internal Docker Compose network; the frontend proxies API calls to the backend under the **/api** path.

EC2 Instance Creation:

Cloud Provider	AWS
Instance Name	Docker-Project-1
Instance ID	i-08f9f985203961d5d
Instance Type	t3.micro (Free Tier Eligible)
Operating System	Amazon Linux 2023 (2023.12.20261102)
Region	Asia Pacific (Mumbai) - ap-south-1
Public IPv4 Address	15.252.183.17
Key Pair (.pem file)	Docker.pem
Status	Running

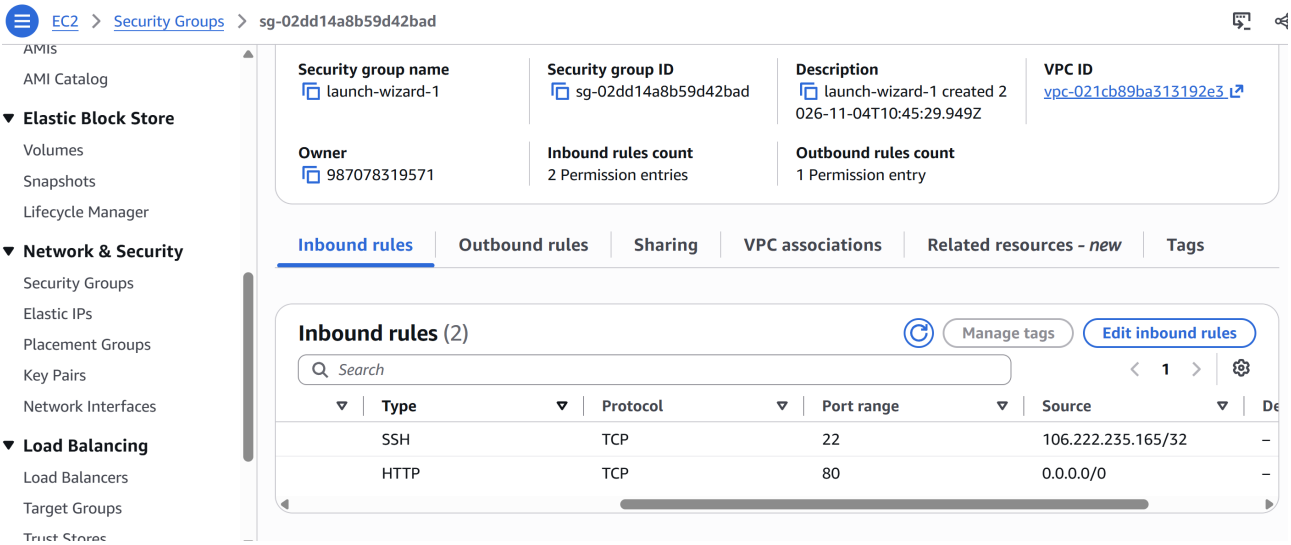


EC2 console: instance Docker-Project-1 in Running state.

Security Group Configuration:

Type	Protocol	Port	Source
SSH	TCP	22	106.222.235.165/32 (my IP only)
HTTP	TCP	80	0.0.0.0/0

SSH is limited to a single IP address. Ports 8000 (backend) and 5432 (PostgreSQL) are intentionally not opened, because those services are only used inside the Docker network.



Security group launch-wizard-1: inbound rules.

Docker Installation and Docker Compose Installation:

Connected to the instance over SSH. Both Docker and Docker Compose were installed successfully on the EC2 instance, and the Docker service is enabled and running.

```
PS C:\Users\pavan\Docker-Project-1> ssh -i "E:\Downloads\Docker.pem" ec2-user@15.252.183.17
    ,     #_
    ~\_  ####_        Amazon Linux 2023
   ~~  \_#####\
   ~~     \###|
   ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
    ~~       V~' '->
     ~~~         /
       ~~._.   _/
          _/ _/
        _/m/'
Last login: Wed Nov  4 11:49:12 2026 from 106.222.235.165
[ec2-user@ip-172-31-12-255 ~]$ cat /etc/os-release
NAME="Amazon Linux"
VERSION="2023"
ID="amzn"
ID_LIKE="fedora"
VERSION_ID="2023"
PLATFORM_ID="platform:al2023"
PRETTY_NAME="Amazon Linux 2023.12.20261102"
ANSI_COLOR="0;33"
CPE_NAME="cpe:2.3:o:amazon:amazon_linux:2023"
HOME_URL="https://aws.amazon.com/linux/amazon-linux-2023/"
DOCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
SUPPORT_URL="https://aws.amazon.com/premiumsupport/"
BUG_REPORT_URL="https://github.com/amazonlinux/amazon-linux-2023"
VENDOR_NAME="AWS"
VENDOR_URL="https://aws.amazon.com/"
SUPPORT_END="2029-06-30"
[ec2-user@ip-172-31-12-255 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-172-31-12-255 ~]$ docker compose version
Docker Compose version v5.7.0
[ec2-user@ip-172-31-12-255 ~]$ sudo systemctl status docker --no-pager
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
     Active: active (running) since Wed 2026-11-04 11:45:22 UTC; 2h 18min ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Process: 29172 ExecStartPre=/bin/mkdir -p /run/docker (code=exited, status=0/SUCCESS)
    Process: 29173 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (code=exited, status=0/SUCCESS)
   Main PID: 29174 (dockerd)
```

SSH login, OS version, Docker 25.0.14, Docker Compose v5.7.0 and docker.service status (active, running).

Project Files on EC2:

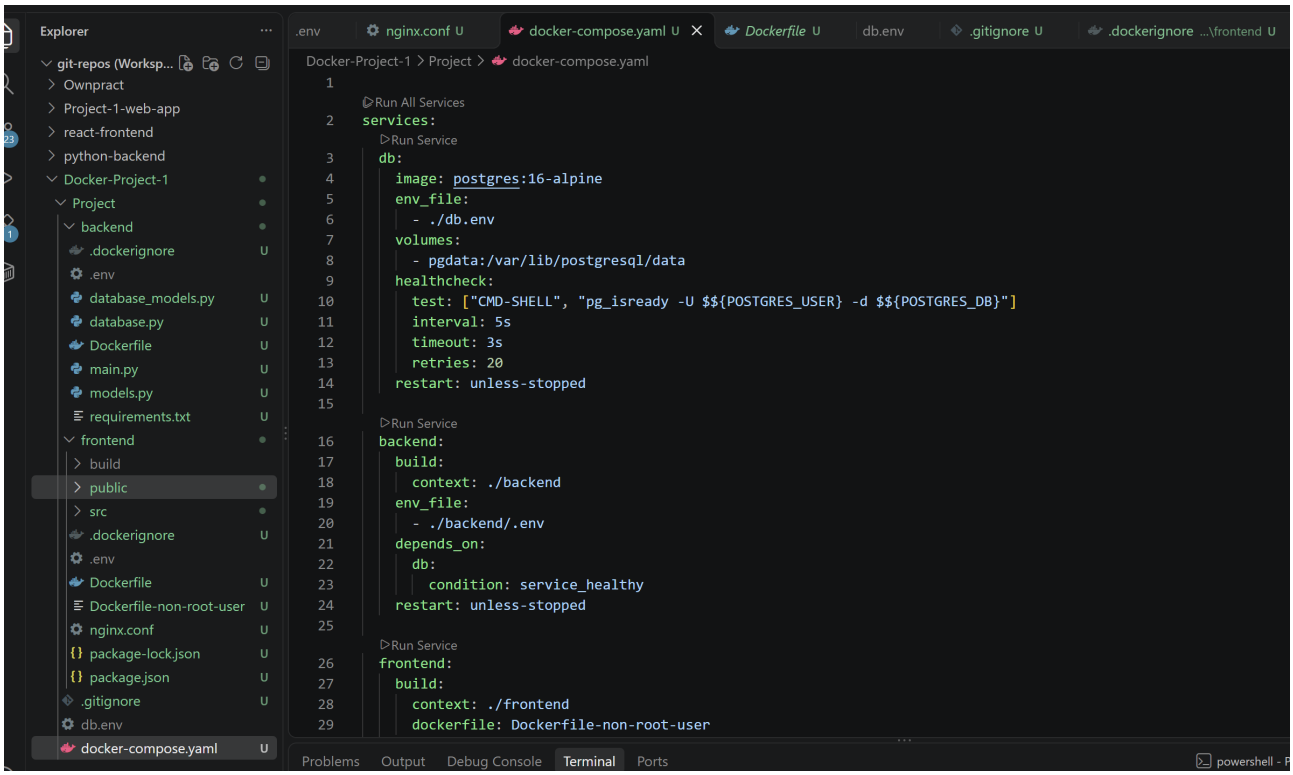
The project directory on the instance contains the backend and frontend folders, db.env, docker-compose.yaml and .gitignore.

```
[ec2-user@ip-172-31-12-255 ~]$ cd ~/Relentless-Tracker
[ec2-user@ip-172-31-12-255 Relentless-Tracker]$ ls -la
total 16
drwxr-xr-x. 4 ec2-user ec2-user 115 Nov  4 12:33 .
drwx------. 5 ec2-user ec2-user 173 Nov  4 12:04 ..
-rw-r--r--. 1 ec2-user ec2-user 222 Nov  4 01:35 .gitignore
drwxr-xr-x. 2 ec2-user ec2-user 158 Nov  4 12:13 backend
-rw-r--r--. 1 ec2-user ec2-user  71 Nov  4 12:34 backocancel
-rw-------. 1 ec2-user ec2-user  70 Nov  4 12:12 db.env
-rw-r--r--. 1 ec2-user ec2-user 724 Nov  4 01:34 docker-compose.yaml
drwxr-xr-x. 4 ec2-user ec2-user 177 Nov  4 12:17 frontend
[ec2-user@ip-172-31-12-255 Relentless-Tracker]$
```

Contents of ~/Relentless-Tracker.

Docker Compose Configuration:

Docker Compose is used to manage all three containers: **db** (postgres:16-alpine with a health check and named volume), **backend** (FastAPI, starts only after the database is healthy) and **frontend** (React build served by Nginx, built from Dockerfile-non-root-user).



docker-compose.yaml and project files.

Running the Application:

The stack is started with **docker compose up -d**. The database reports Healthy first, then the backend and frontend containers start.

```
[+] down 4/4
 ✓ Container relentless-tracker-frontend-1 Removed                                                
```

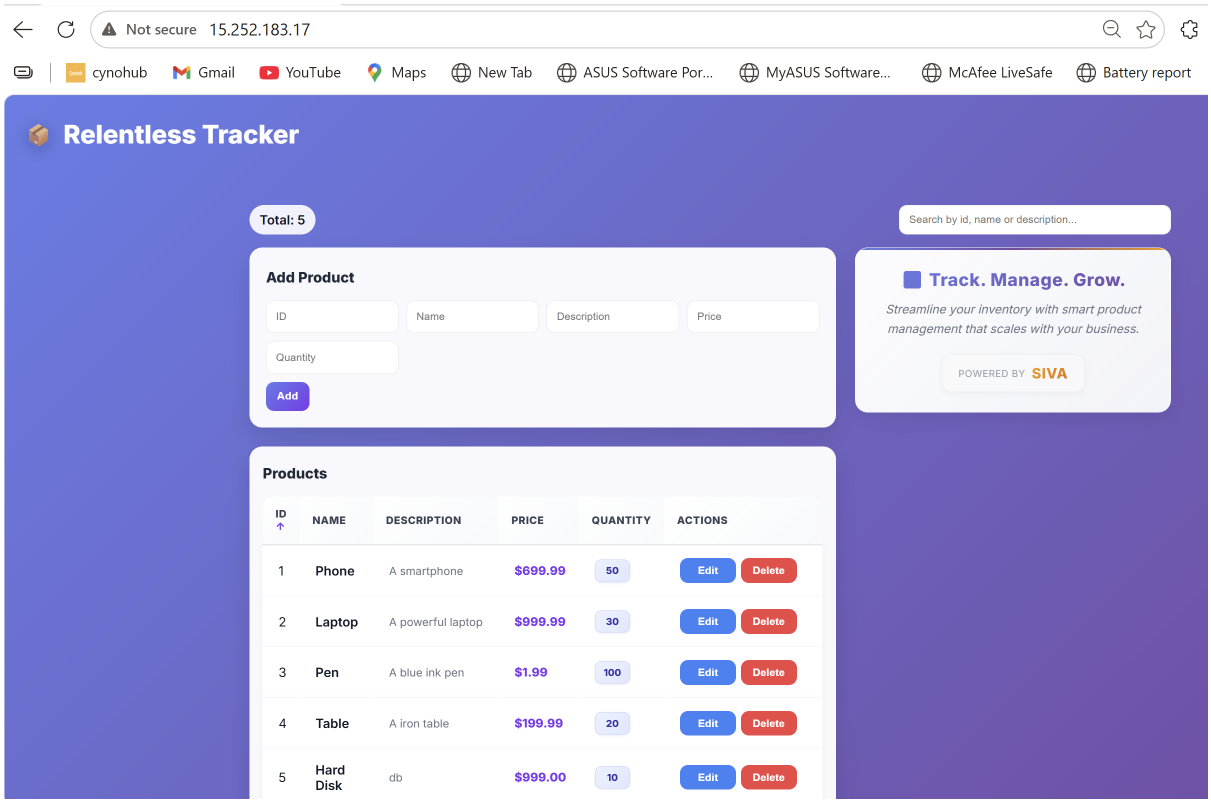
Database Persistence:

PostgreSQL uses a named volume (pgdata) so that data is retained even after restarting the containers.

To verify database persistence, the application was stopped using **docker compose down** and started again using **docker compose up -d**. The containers and the network were removed and recreated, but the named volume was not removed, so the product records stored in PostgreSQL remain available in the application. This confirms that PostgreSQL data is stored in a named Docker volume and survives container restarts.

Frontend Deployment:

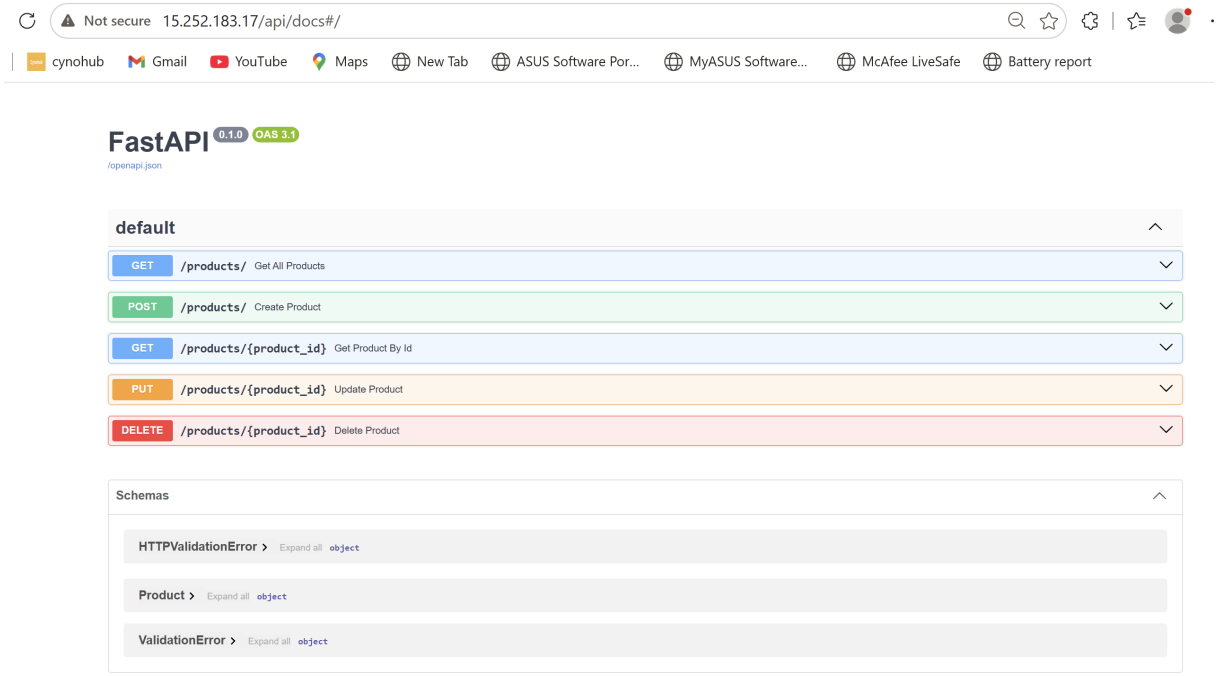
The React frontend is accessible in the browser at the EC2 public IP address (http://15.252.183.17).



Relentless Tracker UI loading product data from the backend and database.

Backend Verification:

The FastAPI Swagger documentation is available at http://15.252.183.17/api/docs and lists the product endpoints (GET, POST, PUT, DELETE).



FastAPI docs served through the frontend proxy at /api/docs.

Conclusion:

Successfully deployed a 3-tier application on AWS EC2 using Docker and Docker Compose. The application is accessible through the EC2 Public IP address and all the project requirements have been satisfied.